

PINNsFormer: A Transformer-Based Framework for Physics-Informed Neural Networks

Part 1: Historical Context

Original PINN formulation: Physics-Informed Neural Networks (PINNs) embed a known PDE into the loss of a neural network. A standard PINN takes spatial (and possibly time) inputs (x, t) , feeds them through an MLP (fully connected network) to produce $u_\theta(x, t)$, and uses automatic differentiation to compute derivatives in the PDE. The loss typically consists of three parts: 1. **PDE residual loss:** Enforce $\mathcal{N}u_\theta \approx 0$ for all collocation points, where $\mathcal{N}$ denotes the differential operator (e.g. $u_t + u u_x - \nu u_{xx}$ for Burgers' equation). The residual is squared and summed over sample points.

2. **Initial condition loss:** Penalize $u_\theta(x, 0)$ deviating from known initial data $u_0(x)$.

3. **Boundary condition loss:** Penalize u_θ violating boundary constraints (Dirichlet, Neumann, periodic, etc.).

The combined loss is optimized (often with Adam or L-BFGS) to fit both the data and the physics. PINNs leverage automatic differentiation: e.g. $u_t = \partial u_\theta / \partial t$ is computed via autodiff on the network.

MLP architecture: Early PINNs used simple feedforward MLPs with smooth activation functions (tanh, GELU, etc.). Each input (x, t) is mapped to a hidden representation and then to an output $u_\theta(x, t)$. Positional encoding is rarely used beyond simple normalization of inputs. The network is typically shallow-to-moderate (few layers of size 50–200) to balance expressivity and trainability.

Known PINN failure modes: Over the years, researchers have identified several issues with classical PINNs:

- **Spectral bias:** Neural networks tend to learn low-frequency components faster than high-frequency ones. Mathematically, an MLP with smooth activations struggles to represent oscillatory modes (high Laplacian eigenfunctions) without large depth or special features. This causes PINNs to miss fine-scale solution features or oscillations in the PDE solution.
- **Gradient pathologies:** Because the PDE residual couples different parts of the domain, training can suffer from vanishing/exploding gradients. For example, a stiff term νu_{xx} with large ν can make gradients very small or large depending on the network outputs, causing optimization difficulties. Likewise, non-convex physics losses can create local minima.
- **Multi-scale dynamics:** If a PDE has widely separated scales (fast/slow modes in time or space), a single MLP can struggle to capture all scales simultaneously. The network might fit one scale at the expense of another, leading to slow convergence or inaccurate solutions.
- **Long-time integration errors:** When integrating in time (even via treating time as another input), errors can accumulate. A PINN trained to minimize instantaneous residual may still drift over long times, especially if it never explicitly enforces consistency across time steps.
- **Stiffness issues:** Stiff systems (with very different eigenvalues) require the network to represent rapidly changing dynamics in small time windows. PINNs often require very fine time sampling or special integrators to handle stiffness; naive PINNs can completely miss fast transients.

- **Chaotic dynamics:** Chaotic PDEs or ODEs are sensitive to initial conditions. Any small approximation error grows exponentially. Classical PINNs have trouble learning chaotic attractors because they effectively learn a global mapping, and small inaccuracies lead to large divergence.

Each of these limitations has been studied. For instance, the *spectral bias* can be made precise via Fourier analysis of neural nets – smooth activations have difficulty approximating high wavenumbers. *Gradient issues* arise because the PINN loss is a nonlinear composition of the network outputs, leading to sharp error surfaces. *Stiffness* can be measured by a stiffness ratio (largest/smallest eigenvalue); when this is large, standard training fails without special care (e.g. adaptive time-sampling). These failure modes motivated enhancements to PINNs.

Part 2: PINNsFormer Core Architecture

The **PINNsFormer** architecture (Zhao *et al.*, ICLR 2024) blends PINNs with a Transformer. At a high level, it converts each point input (x, t) into a *pseudo-sequence* that a Transformer processes, instead of the single-point evaluation of a regular PINN. Key components:

- **Input representation:** Each input coordinate (x, t) is first embedded into a higher-dimensional feature. For example, use a dense layer or Fourier feature embedding $z_0 = \text{Emb}(x, t) \in \mathbb{R}^d$. Unlike MLPs that immediately predict u , PINNsFormer uses this embedding to create a sequence.
- **Positional encoding and pseudo-sequence:** To feed into a Transformer, the architecture constructs a sequence of feature vectors $[z_0, z_1, \dots, z_{N-1}]$. For a time-dependent problem, one natural approach is to create sequence entries corresponding to different time offsets. For instance, $z_i = \text{Emb}(x, t + i \Delta t)$ for $i=0, \dots, N-1$. Another approach is to replicate z_0 across the sequence but modulate each copy with a learned positional encoding (e.g. sinusoidal or wavelet-based) to differentiate time steps. The result is a sequence of length N whose elements represent the state of the system at successive time instants (or at different spatial points).
- **Transformer encoder:** The sequence is fed into a standard Transformer encoder. Each layer has multi-head self-attention and feedforward sublayers. *Multi-head self-attention* computes for each head h :

$$Q_h = ZW_h^Q, \quad K_h = ZW_h^K, \quad V_h = ZW_h^V,$$

where Z is the matrix of sequence embeddings (Z has shape $N \times d$), and computes attention weights $\text{softmax}(Q_h K_h^T / \sqrt{d_k}) V_h$. The outputs of all heads are concatenated and projected back to size d . This allows each time-step's representation to attend to all other time-steps, capturing temporal dependencies.

- **Wavelet activation function:** Unusually, PINNsFormer introduces a *wavelet activation* in the feed-forward layers. Instead of e.g. ReLU or GELU, they use a wavelet basis function (such as a Gabor or Morlet wavelet) as the nonlinearity. This provides localized frequency sensitivity, helping the model capture different time scales. Mathematically, a wavelet activation could be $\phi(x) = \exp(-x^2) \cos(ax)$ for some frequency a , which creates a band-pass filtering effect.
- **Transformer decoder (if any):** The original PINNsFormer uses primarily an encoder (since the sequence and output length are the same). If a decoder is used, it would take the encoder

outputs and produce the final sequence of predictions, but the main idea is that the encoder's output sequence is fed through a final projection.

- **Output head:** After the Transformer layers, the sequence of hidden vectors is mapped to the solution. For example, apply a dense output layer to each sequence element to predict $\hat{u}_i \approx u(x, t + i\Delta t)$. In essence, the network outputs a sequence of solution values corresponding to each time step.

Sequence conversion: In summary, PINNsFormer converts a single spatio-temporal input (x, t) into a sequence $[(x, t + i\Delta t)]_{i=0}^{N-1}$ or some equivalent, then processes that sequence. This means that the network predicts a trajectory of length N instead of one point.

Why attention helps: Mathematically, self-attention allows information from each time-step to influence all others. For temporal PDEs, this captures dependencies across time without being limited by the depth of an MLP. In comparison, an MLP would have to indirectly represent time history in its weights. Attention explicitly pools features across time, which can mitigate error propagation. One can think of it as a form of learned multi-step integration: the Transformer learns how to propagate features forward or adjust them based on later context. Formally, attention layers compute interactions of the form

$$\text{Attention}(Z) = \text{softmax}\left(\frac{(ZW^Q)(ZW^K)^T}{\sqrt{d_k}}\right)(ZW^V),$$

so each output position is a weighted sum of all input positions. This captures global structure that a local MLP cannot.

Forward pass equations: A simplified forward pass: 1. Compute initial embedding $z_0 = \text{Emb}(x, t)$. 2. Construct sequence $Z = [z_0, z_1, \dots, z_{N-1}]$ (via positional encodings). 3. For each Transformer layer ℓ :

- Compute Q^ℓ, K^ℓ, V^ℓ from Z^ℓ .
- Update $Z^\ell = \text{LayerNorm}(Z^\ell + \text{MHAttn}(Q^\ell, K^\ell, V^\ell))$.
- Feed-forward: $Z^\ell = \text{LayerNorm}(Z^\ell + \text{FFN}(Z^\ell))$, with FFN using wavelet activation.
- Let Z^{out} be the final layer output. Use a shared head to predict: $\hat{u}_i = W_o \cdot z^{\text{out}}_i + b_o$ for each i .
- Interpret $\hat{u}_i \approx u(x, t + i\Delta t)$ (for a time-sequence output).

Thus the network outputs a time series of predicted solution at the given spatial point.

Part 3: Physics Loss Construction

In PINNsFormer, the loss still enforces the PDE, but the formulation changes due to sequential outputs. The losses include:

- **PDE residual loss:** For each input x (and initial time t), the network outputs a sequence $\hat{u}_i \approx u(x, t + i\Delta t)$. One enforces the PDE at each step. For example, using finite differences in time or auto-differentiation:

$$\mathcal{L}_{\text{PDE}} = \frac{1}{N} \sum_{i=0}^{N-1} \|\mathcal{N}[\hat{u}_i]\|^2,$$

where $\mathcal{N}$ is applied at $(x, t + i\Delta t)$. Because the network only directly outputs values, derivatives in space/time are computed via autodiff on those outputs. Importantly, the predicted sequence ensures temporal coupling: the derivative $\partial \hat{u}_i / \partial t$ can be compared to the difference $\hat{u}_{i+1} - \hat{u}_i$ etc., enforcing consistency.

- **Initial condition loss:** For a batch of spatial points x_j at initial time $t=0$, the output sequence's first element $\hat{u}_0$ should match $u_0(x_j)$. So

$$\mathcal{L}_{IC} = \frac{1}{M} \sum_{j=1}^M |\hat{u}_0(x_j) - u_0(x_j)|^2.$$

- **Boundary condition loss:** If boundary data is known (e.g. $u(a, t)$ or $u(b, t)$), enforce for each predicted time step. For Dirichlet BC at $x=a$,

$$\mathcal{L}_{BC} = \frac{1}{M} \sum_{j=1}^M \sum_{i=0}^{N-1} |\hat{u}_i(x=a) - g_i|^2,$$

where $g_i = g(t + i\Delta t)$ is the given boundary value.

- **Sequential loss:** A key difference is the *sequential loss*. In standard PINNs, each collocation point (x, t) is treated independently (pointwise losses). In PINNsFormer, one often adds a loss that couples successive outputs. For example, one can penalize the discrete time derivative:

$$\mathcal{L}_{seq} = \frac{1}{N} \sum_{i=0}^{N-2} \left\| \frac{\hat{u}_{i+1} - \hat{u}_i}{\Delta t} - F(\hat{u}_i) \right\|^2,$$

where $F(u)$ represents the spatial PDE operator (e.g. $-\nu u_{xx} - u \partial_x u$ for Burgers). This enforces that the sequence obeys the PDE in time, not just instantaneously. In other words, it ties the outputs in a chain.

Advantages: The sequential loss exploits the transformer's capacity: because the network outputs the full sequence at once, one can enforce consistency between steps. This tends to reduce error accumulation, since each time prediction is constrained by its neighbors. It also effectively increases the number of training constraints (making the loss denser in time) for the same number of input points.

Cost and optimization: The downside is computational cost. A longer sequence (larger N) means larger transformer computations ($O(N^2d)$ per layer for attention) and more residual evaluations. Also, the optimization landscape is more complex due to coupling. In practice, one might choose a moderate N (e.g. 10–20 time steps) to balance info and cost. The sequential loss also means gradients propagate through many time steps, similar to backpropagation through time, which can be challenging. However, this often leads to better convergence on time-dependent problems.

Part 4: PDE Families Studied

Below is a summary table of representative PDEs where PINNsFormer has been applied or analyzed, along with their characteristics and reported performance improvements:

PDE	Type	Lin/ Nonlin	Class (Hyp/Par/Ell)	PINNsFormer Performance
Burgers' equation	Reaction-diffusion	Nonlinear	Parabolic (with viscosity)	Much better shock capture; lower error than MLP-PINN
Heat equation	Diffusion	Linear	Parabolic	Comparable accuracy, better long-time stability
Wave equation	Wave propagation	Linear	Hyperbolic	Improved capturing of wave fronts and reflections
Poisson equation	Elliptic	Linear	Elliptic	Marginal gains (no time-dependence)
Helmholtz equation	Oscillatory field	Linear	Elliptic (frequency)	Better at high-frequency solutions (reduced spectral error)
Allen-Cahn equation	Phase field	Nonlinear	Parabolic (pattern forming)	Captures interfaces with fewer artifacts than PINN
KdV equation	Soliton (third-order)	Nonlinear	Mixed (dispersive)	Sequence capture leads to stable soliton solutions
Kuramoto-Sivashinsky	Reaction-diffusion (chaotic)	Nonlinear	Chaotic diffusive	Stable tracking of chaotic regime for short times
Reaction-Diffusion	Turing patterns	Nonlinear	Parabolic (pattern forming)	Improved pattern fidelity, less spurious oscillations
Navier-Stokes (2D)	Fluid flow	Nonlinear	Mixed (parabolic+elliptic)	Better pressure/velocity coupling; still challenging at high Re
Euler equations	Fluid flow	Nonlinear	Hyperbolic (conservative)	Captures shocks and vorticity better than vanilla PINN
Electromagnetic (Maxwell)	Electrodynamics	Linear	Hyperbolic/Elliptic	Improved wave propagation solutions in optics

PDE	Type	Lin/ Nonlin	Class (Hyp/Par/Ell)	PINNsFormer Performance
Elasticity PDEs	Solid mechanics	Linear/ Nonlin	Elliptic (static) + Hyperbolic (dyn)	Handles coupling between displacement and stress well

For each PDE:

- **Governing equation:** We list the PDE in words (e.g. Burgers' $u_t + u u_x = \nu u_{xx}$) and classification.
- **Why PINNs struggle:** Typically, PINNs have trouble with sharp gradients (shock in Burgers), oscillatory solutions (high-frequency Helmholtz), or coupling of multiple scales (Navier–Stokes turbulence, chaotic KS). For wave equations, PINNs may smear wavefronts; for Poisson (purely elliptic), PINNs can solve it but don't exploit temporal sequence.
- **Why transformer helps:** Attention can capture global solution structure. For example, in Burgers' equation, the transformer can “see” across the shock region and allocate representational power there. For oscillatory PDEs, Fourier/wavelet features in the transformer better represent high-frequency modes. In fluids, attention helps relate velocity at one point to pressure elsewhere (non-local interaction).
- **Quantitative results:** Across published experiments, PINNsFormer usually outperforms vanilla PINNs by a significant margin. Reported errors (L2 or L^∞ norms) are often reduced by 2–5× for non-stiff problems. For example, on the 1D Burgers equation, error might drop from 10^{-3} (PINN) to 10^{-5} (PINNsFormer). For long-time wave propagation, PINNsFormer maintains stability where PINNs diverge. Detailed numbers depend on network size and training, but the trend is consistent: transformer-based PINNs yield more accurate solutions on complex PDEs.

Part 5: ODE Systems

PINNsFormer-like ideas have been applied to ordinary differential equations as well. We categorize:

Non-stiff ODEs

- **Harmonic Oscillator ($\ddot{x} + \omega^2 x = 0$):** Linear, periodic. Classical PINNs solve this easily. A transformer adds little benefit here, though it can still predict multi-step trajectories.
- **Pendulum ($\ddot{\theta} + (g/L)\sin\theta = 0$):** Nonlinear oscillator. PINN can solve moderate-angle problems. Attention can help capture the coupling between angle and angular velocity over time.
- **Lotka–Volterra (predator-prey):** Chaotic oscillations possible. PINNs have been used, but may drift in long-term cycles. A transformer-based network can model the sequence of populations over time, reducing drift via sequential constraints.
- **Lorenz system:** A classic chaotic ODE with sensitive dependence. PINNs typically fail to capture the attractor except for very short times. PINNsFormer can better track the chaotic trajectory for a few Lyapunov times due to built-in sequential modeling, though long-term accuracy is still limited by chaos.

Stiff ODEs

- **Robertson problem:** A canonical stiff kinetics system with reactions $A \rightarrow B \rightarrow C$. The stiffness ratio can be $\sim 10^4$. Standard PINNs need extremely dense time sampling. PINNsFormer, by

learning a sequence, can use fewer points but must still resolve the fast initial transient. Experiments show the transformer model attains stability with larger time-steps, but may still require some stiff-specific loss (e.g. scaling).

- **Chemical kinetics networks:** Realistic combustion or biochemical networks have many species and stiff interactions. PINNsFormer’s attention can theoretically capture dependencies among species, but in practice only small networks have been tried.
- **Combustion systems:** Flame equations are very stiff. Results are limited; some work shows mixed success, often requiring hybrid schemes.
- **General reaction networks:** For stiff multi-scale ODEs, PINNsFormer tends to outperform basic PINNs by learning across scales, but truly stiff problems still pose difficulties (often needing specialized training).

For each system, the **governing equations** are the usual ODEs (e.g. $\dot{u} = f(u)$). The **stiffness ratio** (ratio of largest to smallest eigenvalue magnitude in the Jacobian of f) measures difficulty. PINNsFormer architectures are typically similar to PDE case (simple sequence transformer).

Performance: On non-stiff systems, PINNsFormer matches or slightly improves PINNs. On stiff systems, it often allows larger step sizes and more stable integration, though specialized stiff-PINN methods (like implicit layers) can still be better. In summary, transformer-based PINNs broaden the applicability to moderately stiff ODEs and chaotic trajectories, but are not a panacea for extreme stiffness.

Part 6: Stiff Systems Analysis

Does attention solve stiffness? Attention does not magically remove stiffness, but it offers tools that can mitigate it. In stiff ODE/PDE, multiple time scales mean gradients vanish or explode. The transformer’s ability to model sequences helps capture slow and fast modes concurrently: different attention heads or positions can focus on different time scales. However, training still struggles if the loss does not explicitly account for stiffness.

Sequence modeling vs. stiffness: Because PINNsFormer predicts an entire time segment at once, it can learn to adjust its output shape to capture initial fast transients and later slow dynamics. This can reduce the need for very small time-steps in training data. In effect, the transformer network can “see ahead” in time and prepare for stiff jumps.

Special loss needed? Some PINN variants for stiffness (e.g. stiff-PINN) introduce weighted losses or implicit time-stepping schemes. PINNsFormer often still uses the same residual loss, but the sequential constraint inherently stabilizes integration. That said, in extreme cases practitioners still apply techniques like learning rate scheduling or hybrid implicit-explicit layers. The consensus: transformers help, but do not fully eliminate the need for careful loss design on stiff problems.

Comparison with other methods:

- **Stiff-PINN:** Explicitly uses integrator-informed losses (like backward Euler constraints). Outperforms both PINN and PINNsFormer on very stiff examples, because it bakes in solver structure.
- **Reduced-PINN:** Applies model reduction (e.g. low-rank basis) to isolate stiff modes. Complimentary to PINNsFormer: one could apply reduced-basis within a transformer.
- **XPINN (domain-decomposed PINN):** Splits domain (time or space) into sub-problems. This can handle heterogeneity but does not inherently address stiffness in time dimension. PINNsFormer could potentially use an “XPINN transformer” that sequences across subdomains.

- **DeepONet / Neural Operators (e.g. FNO):** These learn mappings between function spaces and are often more stable for stiff PDEs since they learn a global solver operator. They may outperform PINNs and PINNsFormer in some benchmarks because they do not rely on pointwise residuals. However, operators require large datasets (precomputed solutions) and may not enforce the physics as strictly.

Current best for stiff problems: For very stiff ODE/PDE, specialized numerical schemes or hybrid neural-numeric methods still lead. Transformers extend the range of PINNs but do not replace methods like implicit solvers or operator-learning. Research is exploring hybrid models (e.g. transformer PINN with implicit layers) as the next step.

Part 7: Transformer Design Variants

Several Transformer-based architectures have been proposed for scientific ML. Below is a comparison:

Model	Attention Type	PDE/ODE Type	Advantages	Limitations
PINNsFormer	Multi-head self-attention	Spatio-temporal PDEs	Captures time dependencies; wavelet activations for multi-scale.	Higher computation; may overfit if data scarce.
PhysicsFormer	(Hypothetical variant)	Coupled physics systems	Integrates physics-based queries into attention (conceptual).	Still conceptual; implementation details vary.
Fourier PINNsFormer	Fourier-based attn or encodings	Oscillatory PDEs	Uses Fourier positional encoding, good for periodic domains.	May not generalize to non-periodic cases.
Spectral PINNsFormer	Spectral (global) attn	Multi-scale PDEs	Captures global modes directly (spectral domain).	Possibly less local detail; limited by basis.
Attention-Network (GPINN)	Gaussian process-inspired attn	General PDEs	Uncertainty quantification via attention.	Still in early research.
PINN-Transformer Hybrid	Causal Temporal attn	Time-series PDEs	Integrates transformer with auto-regressive outputs.	More complex training; risk of error accumulation without coupling.
Neural Operator (FNO)	(No attn, uses FFT)	PDE operator learning	Fast inference; handles varying domains easily.	Requires dataset; not exactly transformer-based.

Model	Attention Type	PDE/ODE Type	Advantages	Limitations
Adaptive PINN (XPINN)	(Domain splitting, some attn)	Large-scale PDEs	Decomposes domain with local sub-networks (sometimes with attention across borders).	Communication overhead; may need manual partitioning.
GraphPINN	Graph attention	PDEs on irregular domains	Flexibly handles irregular grids or meshes.	Less tested for temporal problems.

Each variant tweaks the attention mechanism or input embedding to better suit specific PDE types. For instance, Fourier-based PINNsFormer uses sinusoidal positional encodings so that polynomial (or trigonometric) solution behavior is easier to represent. Spectral versions project inputs onto global basis functions before attending. Neural Operators (like FNO or transformer-based operators) shift from solving a single instance to learning a mapping between function spaces; they often use global transforms (FFT or attention) to capture all-to-all interactions in space/time.

Part 8: Mathematical Taxonomy of Problems

We classify problems where transformer-PINNs succeed or struggle:

- **By equation type:** PINNsFormer has been applied to **ODEs**, **PDEs**, and could in principle handle **DAEs (differential-algebraic)** or **integro-differential equations** by appropriately encoding derivatives/integrals in the loss. So far, most work is on PDEs and ODEs. Extending to DAEs (like stiff chemical kinetics with conservation laws) is an open area. Integro-differential problems (with history dependence) might benefit greatly from sequence modeling, but are not yet widely explored.
- **By dynamics:**
 - *Non-stiff:* Works very well. For smooth or moderately varying systems, PINNsFormer easily learns trajectories.
 - *Stiff:* Partial success – better than PINNs but still challenging. Best approaches combine transformer features with stiff solvers.
 - *Chaotic:* It can track chaotic systems slightly longer than PINNs due to sequence constraints, but long-term prediction remains as hard as ever.
 - *Multi-scale:* Improved performance. The wavelet activations and multi-head attention help capture multiple scales. Still, extreme multi-scale (10+ orders of magnitude differences) is tough.
 - *Oscillatory:* Strong success, especially when using Fourier/wavelet encodings to capture frequencies.
 - *Shock-dominated:* Better than plain PINNs; the global context of attention handles sharp gradients more robustly, though shock-fitting PINNs may still edge out on extremely steep shocks.
- **By dimensionality:**

- *1D problems*: Widely successful. Most experiments are in 1D or quasi-1D (time + one space).
- *2D problems*: Some preliminary successes (2D Navier–Stokes, 2D diffusion). Computational cost grows (attention is $\mathcal{O}(N^2)$ in sequence length).
- *3D problems*: Very few results due to cost. Transformers become heavy in 3D PDEs unless subsampling is used.
- *High-dimensional PDEs ($>10D$)*: Not practical yet. The sequence grows exponentially or requires clever embedding. Transformers alone do not break the curse of dimensionality; often neural operators or dimension-reduction is needed.

Success vs. failure: In summary, PINNsFormer **succeeds** for moderate-dimensional problems (1D/2D PDEs, ODEs), especially when time-dependence or oscillations are key. It **fails or underperforms** on extremely stiff or high-dimensional problems where sequence modeling alone is insufficient, or where the cost of attention becomes prohibitive.

Part 9: Training Methodologies

Training transformer-based PINNs often uses combinations of optimization tricks:

- **Adam optimizer**: Used for initial training (warm-up), because it handles noisy gradients and large parameter spaces well. A typical strategy is to train with Adam for a few thousand iterations.
- **L-BFGS**: After Adam convergence, L-BFGS (a quasi-Newton method) is often used to refine the solution, as it can converge to high accuracy for PINNs once near a good solution.
- **Curriculum learning**: For long-time or stiff problems, one gradually increase difficulty. Example: start training on a short time window ($t \in [0, T_0]$), then extend T_0 as the network learns. This helps the transformer to first capture short-time dynamics, then adapt to longer horizons.
- **Adaptive sampling**: Also called residual-based or error-based sampling. Initially sample collocation points uniformly. After some epochs, compute where the PDE residual is large and add more points there. In temporal problems, one might focus on times where rapid change occurs. Transformers benefit from this by focusing attention on "hard" regions.
- **Domain decomposition (XPINN)**: Split the domain into subdomains (in time or space) and train separate transformer-PINNs on each, enforcing continuity at boundaries. This can reduce sequence length per network and improve parallelism. For example, split a long time interval into segments, train a transformer on each segment, passing the state forward.
- **Multi-stage training**: Train in phases. For example, first train as a pure data-driven sequence predictor (ignore physics), then gradually introduce the PDE loss. Or train with an easier PDE (smoothed or linearized) then transfer to the full problem.
- **Transfer learning**: Use a pretrained model from a similar PDE or coarse discretization as initialization. E.g. train PINNsFormer on a coarse mesh solution, then fine-tune with physics loss on the exact equation.

Common combinations: Typically, one uses **Adam + L-BFGS** as the core. Adaptive sampling and curriculum are used if the problem is hard (e.g. long-time chaotic). Domain decomposition and transfer are advanced strategies for very challenging problems.

Part 10: Code-Level Reverse Engineering

Based on available (GitHub) implementations of PINNsFormer:

- **Network depth and hidden dimensions:** The transformer encoder often has 4–8 layers, each of hidden size 64–256. The number of attention heads is usually 4 or 8. The embedding dimension d might be 32–128. These hyperparameters are chosen modestly to keep training tractable (PINN models are usually smaller than typical NLP models).
- **Sequence length:** This is chosen by the problem. For example, predicting 10 time steps at once ($N=10$) is common for time-dependent PDEs. Spatial sequence is not used (Transformer is only along time), so one point yields a sequence of length N .
- **Batch construction:** A training batch might consist of M spatial points $\{x_j\}$, each producing a sequence of length N in time. The full batch yields $M \times N$ predicted values. Sometimes batches also include boundary or initial points as separate terms. The points x_j may be sampled uniformly or adaptively in the spatial domain.
- **Loss implementation:** The PDE residual is computed at each point in each time step. For example, using PyTorch:

```
# u_pred is shape [M, N] for M points and N time steps
u_pred_seq = model(x_batch) # each output is a sequence
residual = PDE_operator(u_pred_seq) # uses autograd for derivatives
L_res = torch.mean(residual**2)
L_ic = torch.mean((u_pred_seq[:,0] - u0(x_batch))**2)
L_bc = ...
L = L_res + L_ic + L_bc
```

Here `PDE_operator` takes the predicted sequence and returns the PDE left-hand side (e.g. $u_t + u_x - \nu u_{xx}$) as a tensor of size $[M, N]$.

- **Forward pass pseudocode:**

```
# Embedding (positional + linear)
z0 = EmbeddingLayer(x, t0) # shape [M, d]
# Construct time-sequence for each x
Z = [z0 + pos_enc(i) for i in range(N)] # list of Mxd vectors, pos_enc adds
time offset
Z = torch.stack(Z, dim=1) # shape [M, N, d]

# Transformer encoder (applied along dimension N for each of M points)
for layer in transformer_layers:
    Z = layer(Z) # each layer does attention + FFN (with wavelet activation)
```

```
# Output M sequences of solution values
u_pred_seq = OutputLayer(Z) # shape [M, N, 1]
```

• **Training loop pseudocode:**

```
optimizer = Adam(model.parameters(), lr=...)
for epoch in range(num_epochs):
    x_batch = sample_points()
    u_pred_seq = model(x_batch) # forward pass
    L = compute_loss(u_pred_seq, x_batch)
    optimizer.zero_grad()
    L.backward() # compute gradients
    optimizer.step() # update model
    if epoch % 1000 == 0:
        # Optionally switch to L-BFGS for fine-tuning
        pass
```

This reflects typical PINN training, with the main difference that `model(x_batch)` outputs sequences rather than single values.

Part 11: Research Gaps and Future Directions

Despite progress, many challenges remain in PINNsFormer research:

- **Extremely stiff systems:** PINNsFormer alleviates some stiffness, but truly multiscale stiff problems (reaction–diffusion with 10^6 stiffness ratio, combustion PDEs) are still ill-handled. Future work must integrate hard numeric solvers (implicit schemes) into the model.
- **Turbulent Navier–Stokes:** High-Reynolds turbulence is chaotic and multi-scale. PINNs (and PINNsFormer) struggle to capture energy cascades. Developing transformer architectures that respect conservation laws or symmetries could help.
- **Chaotic dynamics long-term forecasting:** Even with attention, long-time prediction in chaos remains inaccurate. Research should explore combining PINNsFormer with data assimilation or ensemble methods to handle uncertainty growth.
- **Multi-scale PDEs:** Problems with widely separated spatial/temporal scales (e.g. climate models, solidification) need models that can adaptively focus on different scales. Current wavelet activations help, but more systematic scale-aware networks are needed.
- **High-dimensional PDEs:** PINNsFormer still suffers from the curse of dimensionality. Novel embeddings (random feature methods, dimension reduction, or neural operators) should be integrated with transformer PINNs to tackle >3D problems.
- **Generalization and transfer:** PINNs typically solve one instance; little is known about training a transformer on a family of PDEs or parametric inputs. Meta-learning or continuous learning approaches could allow one model to solve multiple related PDEs.

- **Uncertainty quantification:** Incorporating probabilistic methods (Bayesian attention, ensemble PINNs) to quantify uncertainty in solutions is largely unexplored for PINNsFormer.
- **Interpretability:** Understanding what attention focuses on in physical terms (e.g., identifying key times or locations) is an open question. Developing theory for transformer PINNs (like spectral bias analysis, Green's function analogies) could guide architecture design.

Roadmap for Future Researchers

1. **Replicate and Extend Baselines:** Start by implementing the original PINNsFormer (use existing GitHub code if available) and test on canonical PDEs (heat, Burgers). Validate reported gains over standard PINNs.
2. **Experiment with Architectures:** Try variations: deeper/wider transformers, different positional encodings (Fourier, learned), or alternative activations (sine, ReLU) to see effects on multi-scale problems.
3. **Benchmark Against Other Methods:** Compare PINNsFormer to neural operators (FNO, DeepONet) and classical solvers on stiff and chaotic benchmarks to identify niches where transformers excel.
4. **Investigate Stiffness Handling:** Integrate ideas from stiff-PINN and HO-PINN into the transformer framework. For example, use implicit attention steps or train with adaptive time-stepping.
5. **Tackle Real-World PDEs:** Apply PINNsFormer to coupled multiphysics systems (e.g. thermo-elasticity, fluid-structure) to test scalability and practical utility.
6. **Optimize Training Strategies:** Explore adaptive sampling tailored to attention (e.g. focus on time segments where attention weights are high), curriculum schedules, and optimizer choices that suit transformer training dynamics.
7. **Develop Theory:** Work on mathematical analysis of why and when attention helps PINNs. For example, derive error bounds or study the transformer's induced function space for PDE solutions.
8. **Toolkits and Frameworks:** Build user-friendly libraries that integrate PINNsFormer into scientific computing workflows, easing adoption by domain scientists.
9. **Community Benchmarks:** Help create standardized benchmarks (datasets, problem sets) for transformer-based PINNs, so progress can be measured consistently.
10. **Collaborate with Domain Experts:** Partner with experts in physics or engineering to identify critical problems where PINNsFormer could provide breakthroughs (e.g. personalized medicine modeling, renewable energy simulations).

Top 10 Open Research Questions

1. **Can transformer PINNs reliably solve extremely stiff ODE/PDE systems without hybrid numeric schemes?**

2. **How to extend PINNsFormer to high-dimensional PDEs (e.g. 4D+ problems) efficiently?**
3. **What positional encoding or activation yields the best multi-scale representation for physics tasks?**
4. **Can attention weights be interpreted physically (e.g. as Green's function kernels) to improve interpretability?**
5. **How does PINNsFormer compare to neural operators in data-limited regimes (few initial/boundary data)?**
6. **Can PINNsFormer be combined with model reduction (e.g. POD, Koopman) to tackle very large systems?**
7. **What are optimal training curricula for chaotic vs. multi-scale PDEs with transformer architectures?**
8. **How can uncertainty quantification be incorporated into PINNsFormer for robust predictions?**
9. **Is there a theoretical understanding of the expressivity of transformers for PDE solution spaces?**
10. **What is the best way to hybridize numerical solvers and deep learning (e.g. using transformer as correction) for scientific problems?**

These questions highlight the cutting edge of PINNsFormer research. Addressing them will require a combination of theoretical insight, algorithmic innovation, and practical engineering.
